DRIP

Fashion Reels Platform

Backend API System

SOFTWARE REQUIREMENTS SPECIFICATION

Version 1.0 | April 2026 | Status: Complete

Document Type	Software Requirements Specification (SRS)
Project	Drip — Fashion Reels Platform
Module	Backend REST API
Version	1.0
Date	April 2026
Status	Production Ready
Total Endpoints	80+ REST API Endpoints
Total Phases	13 Complete Backend Phases

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and detailed description of the Drip Fashion Reels Platform Backend API. It covers functional requirements, non-functional requirements, system architecture, database design, API standards, security specifications, AI algorithm design, and real-time system design.

This document serves as the primary technical reference for development, quality assurance testing, and future system maintenance and enhancement.

1.2 Project Scope

Drip is a mobile-first fashion discovery and e-commerce platform. The core concept combines the reels-style short-video format popularized by TikTok with direct in-video shopping, social engagement features, and AI-driven personalization.

The backend system provides REST API services for three distinct user types:

- Users (shoppers) who browse, engage with, and purchase fashion items
- Fashion Partners (brands/sellers) who upload content and fulfill orders
- Administrators who manage the platform, approve partners, and moderate content

1.3 Definitions and Abbreviations

Term / Abbreviation	Definition

User	Registered shopper who browses, likes, bookmarks, comments, and purchases outfits
Fashion Partner	Brand or seller who uploads outfit reels and manages orders
Admin	Platform staff member with management and moderation access
Superadmin	Highest-privilege admin who can create other admins and modify permissions
Outfit Reel	Short video (uploaded to ImageKit CDN) showcasing a fashion item available for purchase
categoryScores	MongoDB Map field on User storing weighted engagement scores per fashion category
relevanceScore	AI-computed score (0–100) representing an outfit's fit for a specific user
conversationId	Unique chat thread identifier generated by sorting two party IDs and joining with underscore
JWT	JSON Web Token — used for stateless authentication
CDN	Content Delivery Network — ImageKit used for video and image hosting
RBAC	Role-Based Access Control — permission system used in Admin panel
Fire-and-Forget	Async operation that does not block the main response if it fails
Soft Delete	Marking a record as inactive (isActive: false) rather than permanently removing it
Cold Start	Problem where a new user has no history for AI to generate recommendations from
DXA	Document eXtension unit (1/1440 of an inch) used in DOCX formatting

1.4 Document Overview

This SRS is organized into 12 sections covering all aspects of the backend system. Sections 3–5 define what the system must do. Sections 6–11 define how it is designed and built. Section 12 lists constraints and assumptions.

2. System Overview

2.1 System Context

The Drip backend API is a Node.js/Express application connecting a React frontend to MongoDB Atlas, ImageKit CDN, and Stripe Payments. Real-time features are powered by Socket.io running on the same HTTP server instance.

Component	Technology	Purpose
Web Server	Node.js 18+ / Express 4.x	API request handling, middleware pipeline
Database	MongoDB Atlas / Mongoose 7.x	Primary data store — all collections
Media CDN	ImageKit	Video and image upload, storage, and delivery
Payments	Stripe	Payment intent creation, webhook processing
Real-Time	Socket.io 4.x	Notifications, chat, typing indicators
Authentication	JWT + bcryptjs	Stateless auth with refresh token rotation
Logging	Winston	Structured logging to console and file
Process Manager	PM2 (recommended)	Production process management

2.2 User Roles and Responsibilities

User (Shopper)

- Browse outfit reels in a personalized vertical feed
- Like, bookmark, comment on outfits
- Follow fashion partner brands
- Add items to cart and place orders (Stripe or COD)
- Chat with fashion partners
- Receive AI-personalized feed based on interaction history
- Complete style quiz for onboarding personalization

Fashion Partner (Brand/Seller)

- Register and await admin approval before login is permitted
- Upload outfit videos and product images to ImageKit CDN
- Create, update, and delete outfit listings
- Receive and update order statuses through defined lifecycle
- Chat with customers regarding orders and sizing

- View own sales analytics and follower counts

Admin (Platform Staff)

- Approve or reject fashion partner registration applications
- Ban and unban user and partner accounts
- Remove policy-violating outfits and comments
- View comprehensive platform analytics
- Manage orders across all partners

Superadmin

- All Admin capabilities plus:
- Create new admin accounts with specific permissions
- Modify existing admin permissions
- Cannot modify own permissions (security constraint)

2.3 System Constraints Overview

Constraint	Value	Rationale
Max images per outfit	5	Storage and performance optimization
Max tags per outfit	10	Search index efficiency
Max cart quantity per item	10	Prevent stock hoarding
Comment nesting depth	1 level	UX simplicity, prevents infinite nesting
Message delete window	24 hours	Industry standard (WhatsApp model)
AI feed pool size	200 outfits	Balance between personalization quality and performance
Access token expiry	15 minutes	Security best practice
Refresh token expiry	7 days	Balance between UX and security
Max file size — Image	5 MB	Page load performance
Max file size — Video	100 MB	Upload time and CDN cost

3. Functional Requirements

3.1 Authentication System

FR-AUTH-01 System shall provide completely separate authentication flows for Users, Fashion Partners, and Admins, each with independent JWT tokens containing role claims.

FR-AUTH-02 Access tokens shall expire in 15 minutes. Refresh tokens shall expire in 7 days and be stored as httpOnly cookies.

FR-AUTH-03 Refresh token rotation shall be implemented. Any reuse of an invalidated refresh token shall immediately invalidate all sessions for that account.

FR-AUTH-04 Passwords shall meet minimum requirements: 8+ characters, at least one uppercase letter, one lowercase letter, one digit, and one special character (@$!%*?&).

FR-AUTH-05 Fashion Partner accounts require explicit admin approval (isApproved: true) before login returns a token. Unapproved partners receive HTTP 403.

FR-AUTH-06 All three account types shall support profile update and password change via separate protected endpoints.

FR-AUTH-07 Banned accounts (isActive: false) shall receive HTTP 403 on login. Existing valid tokens shall also be rejected by auth middleware.

3.2 Outfit Reel Management

FR-OUTFIT-01 Fashion Partners shall upload outfit videos and images to ImageKit CDN via dedicated upload endpoints before creating outfit listings. The API returns CDN URLs and fileIds.

FR-OUTFIT-02 Outfit listings shall contain: title (3–100 chars), description (max 500), video URL + fileId + thumbnailUrl, images array (max 5), price (non-negative), sizes array (min 1), colors array, category (from enum), tags array (max 10, lowercase), stock quantity.

FR-OUTFIT-03 System shall provide a paginated feed endpoint (GET /api/outfit/feed) supporting filters: category, price range (minPrice, maxPrice), and 6 sort options: newest, oldest, price_asc, price_desc, popular (by likes), trending (by views).

FR-OUTFIT-04 viewsCount shall increment on each GET /api/outfit/:id request using the fire-and-forget pattern via MongoDB $inc operator. This shall not block the API response.

FR-OUTFIT-05 discountPercentage and isInStock shall be computed as Mongoose virtual fields, included in toJSON and toObject output.

FR-OUTFIT-06 Outfit deletion shall trigger deletion of the video and all associated images from ImageKit CDN using stored fileIds. DB deletion and CDN deletion shall both be attempted.

FR-OUTFIT-07 MongoDB compound text index shall be created on title, description, tags, and category fields to support full-text search.

FR-OUTFIT-08 Only the partner who created an outfit may update or delete it. Attempting to modify another partner's outfit shall return HTTP 403.

3.3 Social Features

FR-SOCIAL-01 Like/Unlike shall be implemented as a toggle. A compound unique index on {user, outfit} shall prevent duplicate likes at the database level.

FR-SOCIAL-02 likesCount on OutfitItem shall be atomically updated using MongoDB $inc operator on like/unlike to prevent race conditions.

FR-SOCIAL-03 Bookmark/Unbookmark shall follow the same toggle and atomic update pattern as likes, maintaining bookmarksCount on OutfitItem.

FR-SOCIAL-04 Follow/Unfollow shall be a toggle. followersCount on FashionPartner shall be atomically updated. It shall never go below zero.

FR-SOCIAL-05 Comments shall support exactly one level of nesting. Attempting to reply to a reply shall return HTTP 400 with message 'Cannot reply to a reply. Max depth is 1 level.'

FR-SOCIAL-06 Comment deletion shall be a soft delete (isActive: false). Deleting a parent comment shall cascade soft-deletion to all child replies. commentsCount on OutfitItem shall be decremented for parent + all affected replies.

FR-SOCIAL-07 Every social action (like, comment, reply, follow) shall trigger an asynchronous notification to the affected party using the fire-and-forget pattern.

FR-SOCIAL-08 GET /api/social/status/:outfitId shall check both like and bookmark status for the current user in a single parallel Promise.all call.

3.4 Search and Discovery

FR-SEARCH-01 Full-text search (GET /api/search?q=) shall use MongoDB $text operator with textScore for relevance sorting when a keyword is provided.

FR-SEARCH-02 Search shall support simultaneous combination of filters: keyword (q), category, minPrice, maxPrice, size, and partner ID.

FR-SEARCH-03 Trending tags (GET /api/search/trending) shall be computed via aggregation: trendScore = count + (totalViews/10) + (totalLikes/5), limited to last 7 days, with all-time fallback.

FR-SEARCH-04 Autocomplete suggestions (GET /api/search/suggestions?q=) shall require minimum 2 characters, run parallel regex queries on titles and tags, and return results labeled by type (outfit/tag).

FR-SEARCH-05 Category browse (GET /api/search/category/:category) shall return outfits plus aggregated category statistics: totalOutfits, avgPrice, minPrice, maxPrice, totalViews, totalLikes.

FR-SEARCH-06 Personalized feed (GET /api/search/personalized) shall incorporate user's stylePreferences and followed partner IDs. It shall fall back to newest outfits for users with no preferences.

3.5 Cart and Order Management

FR-CART-01 Each user shall have exactly one cart document, enforced by a unique index on the user field. Non-existent carts shall be auto-created on first GET /api/cart request.

FR-CART-02 Price shall be locked at cart addition time in the priceAtAdd field. Subsequent partner price changes shall not affect items already in the cart.

FR-CART-03 An outfitSnapshot (title, thumbnailUrl, category, partnerName) shall be saved in each cart item at the time of addition to preserve display data if the outfit is later deleted.

FR-CART-04 Cart totalAmount shall be auto-calculated in a Mongoose pre-save middleware hook using priceAtAdd × quantity for each item. No manual totalAmount updates are permitted.

FR-CART-05 Item quantity shall be capped at 10 per product. Adding the same outfit+size combination shall merge quantities rather than create a duplicate entry.

FR-CART-06 Stock availability shall be validated on both add-to-cart and at checkout. Insufficient stock shall return HTTP 400.

FR-ORDER-01 Order numbers shall be auto-generated in the Mongoose pre-save hook following the format: DRP-YYYYMM-XXXXXX (6-digit random suffix).

FR-ORDER-02 For Stripe payments: a PaymentIntent shall be created on POST /api/order/checkout and the clientSecret returned. Order status remains 'pending' until the payment_intent.succeeded webhook event is received.

FR-ORDER-03 COD (Cash on Delivery) orders shall be created immediately with paymentStatus: 'unpaid' and do not require Stripe integration.

FR-ORDER-04 Partner shall advance order status only through allowed transitions: confirmed → processing → shipped → delivered. Any other transition shall return HTTP 400.

FR-ORDER-05 Users may cancel orders only in 'pending' or 'confirmed' status. Cancellation of shipped or delivered orders shall return HTTP 400 with support contact information.

FR-ORDER-06 Cart shall be cleared (items: []) immediately after a successful POST /api/order/checkout, regardless of payment method.

3.6 Real-Time Notifications

FR-NOTIF-01 All notifications shall be simultaneously persisted to the notifications MongoDB collection AND delivered in real-time via Socket.io to the recipient's personal room.

FR-NOTIF-02 System shall support 11 notification types: like, comment, reply, follow, new_outfit, order_placed, order_confirmed, order_shipped, order_delivered, order_cancelled, system.

FR-NOTIF-03 Notification creation and delivery failures shall never block or affect the main API operation that triggered them. The fire-and-forget pattern is mandatory for all notification calls.

FR-NOTIF-04 Bulk new-outfit notifications (to all followers of a partner) shall use Promise.allSettled to ensure partial failures do not prevent delivery to other followers.

FR-NOTIF-05 GET /api/notification/unread-count shall return the count of unread notifications for the current user or partner, used for the notification bell badge in the frontend.

FR-NOTIF-06 Marking all notifications as read (PATCH /api/notification/read-all) shall be idempotent and return modifiedCount indicating how many were actually updated.

3.7 Real-Time Chat / Direct Messaging

FR-CHAT-01 conversationId shall be computed by sorting two party ObjectIds as strings alphabetically and joining with underscore. This ensures the same conversationId regardless of who initiates the conversation.

FR-CHAT-02 When a user or partner fetches message history (GET /api/chat/:otherPartyId), all messages addressed to them in that conversation shall be automatically marked as read (isRead: true, readAt: timestamp).

FR-CHAT-03 Read receipt event ('messages_read') shall be emitted via Socket.io to the sender's personal room when the receiver fetches messages.

FR-CHAT-04 Outfit sharing in chat shall save an outfitSnapshot (title, thumbnailUrl, price, category) in the message document to preserve display even if the outfit is later deleted.

FR-CHAT-05 Message deletion shall be a soft delete: isDeleted set to true, text replaced with 'This message was deleted'. The deletion event shall be emitted via Socket.io to the other party.

FR-CHAT-06 Messages may only be deleted within 24 hours of the original send time. Attempting deletion after this window shall return HTTP 400.

FR-CHAT-07 Typing indicators shall be implemented via Socket.io client-emitted events ('typing', 'stop_typing') that the server forwards to the appropriate receiver's room.

3.8 Admin Panel

FR-ADMIN-01 Admin accounts shall have granular permissions: manage_users, manage_partners, manage_content, manage_orders, view_analytics, manage_admins. Each protected route shall verify the required permission.

FR-ADMIN-02 Partner approval workflow: Partner registers (isApproved: false) → Admin reviews → Admin approves → Partner can now login. Rejection shall set isActive: false.

FR-ADMIN-03 Platform analytics (GET /api/admin/analytics) shall compute in a single endpoint using Promise.all: overview counts, thisMonth stats, top 5 outfits by views, top 5 partners by followers, orders grouped by status, revenue grouped by month (last 6).

FR-ADMIN-04 Admin outfit removal shall delete the outfit from MongoDB AND delete the video and all images from ImageKit CDN. The fashion partner shall receive a system notification.

FR-ADMIN-05 Only accounts with adminRole: 'superadmin' may create new admin accounts or modify other admins' permissions. Superadmin cannot modify their own permissions.

3.9 AI Recommendation System

FR-AI-01 The categoryScores Map on each User document shall be updated via MongoDB $inc on every user interaction, using defined weights: view(0.5), like(2.0), comment(2.5), bookmark(3.0), share(3.5), order(5.0).

FR-AI-02 Outfit relevance score (0–100) shall be computed from 6 factors: category score match (max 40), followed partner bonus (max 20), tag overlap with stylePreferences (max 15), direct category preference (max 10), trending score (max 10), freshness bonus (max 5).

FR-AI-03 The personalized feed endpoint shall fetch a pool of up to 200 active outfits, score each using calculateOutfitScore(), sort by relevanceScore descending, then apply pagination on the sorted array.

FR-AI-04 The style quiz endpoint (POST /api/ai/style-quiz) shall map quiz answers to initial categoryScores and seed the user's profile, solving the cold-start problem for new users.

FR-AI-05 Complete the Look (GET /api/ai/complete-look/:outfitId) shall use a hardcoded category complementary map to suggest outfits from compatible categories (e.g., casual → accessories, footwear).

FR-AI-06 Trending outfits (GET /api/ai/trending) shall use the formula: trendScore = (views×0.3) + (likes×0.5) + (bookmarks×0.4) + (comments×0.3), filterable by period: 24h, 7d, 30d.

4. Non-Functional Requirements

4.1 Performance

NFR-PERF-01 Standard GET endpoints shall respond in under 500ms under normal load conditions on a properly sized server instance.

NFR-PERF-02 The AI feed endpoint shall fetch a maximum pool of 200 outfits. Scoring is performed in memory (calculateOutfitScore is a pure function with no database calls).

NFR-PERF-03 All endpoints with multiple independent database queries shall use Promise.all for parallel execution. Sequential awaits on unrelated queries are not permitted.

NFR-PERF-04 View count increments shall use fire-and-forget (not awaited) to avoid adding any latency to the GET /api/outfit/:id response.

NFR-PERF-05 Aggregation pipelines shall use appropriate MongoDB indexes. The analytics endpoint shall complete within 2 seconds on a dataset of 100,000 orders.

4.2 Scalability

NFR-SCALE-01 All list endpoints shall implement cursor or page-based pagination. Maximum page size shall be enforced server-side regardless of client-requested limit.

NFR-SCALE-02 Socket.io shall use room-based event targeting (io.to(room).emit) rather than broadcasting to all connected clients.

NFR-SCALE-03 MongoDB indexes shall be defined for all fields used in WHERE clauses, sort operations, and join conditions. Compound indexes for common query patterns.

4.3 Security

NFR-SEC-01 All endpoints except explicitly public ones shall require JWT authentication. Public endpoints are: feed, single outfit, partner public profile, search, trending, category browse, complete-look, similar outfits.

NFR-SEC-02 Rate limiting shall be applied with different thresholds per route sensitivity level. Responses to rate-limited requests shall include retryAfter information.

NFR-SEC-03 The password field and refreshToken field shall have select: false in their Mongoose schema definitions. They shall never appear in any API response.

NFR-SEC-04 All user-provided text inputs shall be sanitized against NoSQL injection (express-mongo-sanitize) and XSS attacks (.escape() on text fields).

NFR-SEC-05 CORS shall only allow origins listed in the server whitelist. Requests from unlisted origins shall be rejected.

4.4 Reliability

NFR-REL-01 Server shall handle process.on('unhandledRejection') and process.on('uncaughtException') with logged errors and graceful process.exit(1).

NFR-REL-02 Notification, tracking, and socket emission failures shall be logged as warnings but shall never propagate to affect the main API response.

NFR-REL-03 MongoDB connection failure at startup shall log the error and call process.exit(1). The server shall not start in a degraded state without a database connection.

4.5 Maintainability

NFR-MAINT-01 All constants (categories, sizes, currencies, statuses, weights, limits) shall be defined in utils/constants.js. Hardcoded values in controllers are not permitted.

NFR-MAINT-02 All asynchronous controller functions shall be wrapped in the catchAsync utility. Try-catch blocks inside controllers are not permitted.

NFR-MAINT-03 All source files shall begin with a JSDoc @file block. All exported functions shall have JSDoc comments with @param and @returns documentation.

NFR-MAINT-04 Winston logger shall be used for all application logging. console.log, console.error, and console.warn are not permitted in production code.

5. System Architecture

5.1 Request Lifecycle

Every incoming HTTP request passes through the following middleware pipeline in strict order:

Stage	Middleware / Component	Action
1	Rate Limiter	Check request count per IP — return 429 if exceeded
2	Helmet	Attach 15+ security headers to response
3	CORS	Validate origin against whitelist — reject if not allowed
4	Body Parser	Parse JSON body (limit: 10MB)
5	Cookie Parser	Parse cookies for refresh token extraction
6	Morgan	Log HTTP method, URL, status, duration (dev only)
7	MongoSanitize	Strip $ and . operators from request body and params
8	HPP	Remove duplicate query parameters
9	Route Matching	Match URL to registered route handler
10	Auth Middleware	Verify JWT — attach req.user / req.partner / req.admin
11	Permission Check	Verify RBAC permissions for admin routes
12	Validation	Run express-validator chains — return 422 if invalid
13	Controller	Execute business logic wrapped in catchAsync
14	Database	Mongoose operations on MongoDB Atlas
15	Side Effects	Fire-and-forget: notifications, AI tracking, socket events
16	sendResponse	Format and send standardized JSON response
17	Error Middleware	Catch any errors — format and send error response

5.2 Project Folder Structure

Path	Contents
server.js	HTTP server creation, Socket.io initialization, PORT listener, graceful shutdown

app.js	Express app, all middleware stack, route mounting
config/db.js	MongoDB connection with retry logic and event listeners
config/imagekit.js	ImageKit SDK initialization
config/logger.js	Winston logger with file and console transports
config/socket.js	Socket.io server setup, auth middleware, room management, chat events
config/security.js	CORS options, Helmet config, cookie options, env validation
models/ (13 files)	All Mongoose schemas: User, FashionPartner, OutfitItem, Like, Bookmark, Follow, Comment, Cart, Order, Notification, Message, Admin — plus virtual fields, indexes, pre-save hooks
routes/ (13 files)	Express Router definitions with validation chains and middleware composition
controllers/ (13 files)	Business logic functions, all wrapped in catchAsync
middleware/authUser.middleware.js	JWT verification for User routes
middleware/authPartner.middleware.js	JWT verification for Partner routes
middleware/authAdmin.middleware.js	JWT verification + permission check for Admin routes
middleware/upload.middleware.js	Multer memory storage, file type filter, size limits
middleware/error.middleware.js	Centralized error handler — maps error types to HTTP codes
middleware/validate.middleware.js	validateRequest, validateObjectId, reusable field validators
middleware/rateLimit.middleware.js	All rate limiter instances: global, auth, upload, search, AI, strict
services/imagekit.service.js	uploadToImageKit, deleteFromImageKit, getImageKitUrl
services/stripe.service.js	createPaymentIntent, retrievePaymentIntent, constructWebhookEvent, createRefund
services/socket.service.js	createAndSendNotification, emitToRoom, all notification helper functions
services/ai.service.js	trackInteraction, calculateOutfitScore, analyzeUserStyle, getComplementaryOutfits
utils/AppError.js	Custom error class extending Error with statusCode and isOperational

utils/catchAsync.js	Async wrapper that passes rejections to Express error middleware
utils/sendResponse.js	Standardized JSON response formatter
utils/pagination.js	getPagination utility — extracts page, limit, skip from query string
utils/constants.js	All enums, limits, weights, messages — single source of truth
scripts/seedAdmin.js	One-time superadmin seed script (npm run seed:admin)

6. Database Design

6.1 Collections Overview

Collection	Model	Primary Purpose	Key Constraint
users	User.js	Registered shoppers	email unique
fashionpartners	FashionPartner.js	Brand/seller accounts	email unique, brandName unique
outfititems	OutfitItem.js	Outfit reels	Text index on title,description,tags,category
likes	Like.js	User likes on outfits	{user,outfit} compound unique
bookmarks	Bookmark.js	User saved outfits	{user,outfit} compound unique
follows	Follow.js	User follows partner	{follower,following} compound unique
comments	Comment.js	Comments + replies	parentComment ref for nesting
carts	Cart.js	Shopping cart	user unique (one cart per user)
orders	Order.js	Purchase orders	orderNumber unique, DRP-YYYYMM-XXXXXX format
notifications	Notification.js	In-app notifications	{recipient,isRead} compound index
messages	Message.js	Direct messages	{conversationId,createdAt} compound index
admins	Admin.js	Platform administrators	email unique, separate from users

6.2 Critical Database Indexes

Collection	Index Fields	Type	Purpose
outfititems	title, description, tags, category	Text	Full-text search
outfititems	isActive, createdAt	Compound	Feed query performance
outfititems	partner, isActive	Compound	Partner outfits listing
likes	user, outfit	Unique Compound	Prevent duplicate likes

bookmarks	user, outfit	Unique Compound	Prevent duplicate bookmarks
follows	follower, following	Unique Compound	Prevent duplicate follows
comments	outfit, createdAt	Compound	Comment listing by outfit
comments	parentComment	Single	Reply lookup
messages	conversationId, createdAt	Compound	Chat history pagination
messages	receiver, isRead	Compound	Unread message count
notifications	recipient, isRead	Compound	Unread notification query
notifications	recipient, createdAt	Compound	Notification listing
orders	user, createdAt	Compound	User order history
orders	partner, status	Compound	Partner order management
orders	orderNumber	Unique	Order lookup by number

7. API Design Standards

7.1 Standard Response Format

All API responses — success and error — shall conform to the following JSON structure:

Field	Type	Always Present	Description
success	Boolean	Yes	true for 2xx responses, false for 4xx/5xx
message	String	Yes	Human-readable description of the result
data	Object|null	Yes	Response payload — null for deletions and logouts
pagination	Object|null	Conditional	Present on all list endpoints
errors	Array|null	Conditional	Present on 422 validation failures — field-level errors

7.2 Pagination Object

Field	Type	Description
page	Integer	Current page number (1-indexed)
limit	Integer	Items per page (server-enforced maximum)
total	Integer	Total number of matching records
totalPages	Integer	Ceiling(total / limit)
hasNextPage	Boolean	True if page < totalPages
hasPrevPage	Boolean	True if page > 1

7.3 HTTP Status Code Usage

Code	Status	When Used
200	OK	Successful GET, PATCH, DELETE — resource found and processed
201	Created	Successful POST — new resource created
400	Bad Request	Business logic violation (e.g., cancelling shipped order, empty cart checkout)

401	Unauthorized	Missing, invalid, or expired JWT token
403	Forbidden	Valid token but insufficient role or permission
404	Not Found	Requested resource does not exist or does not belong to requester
409	Conflict	Duplicate unique field (email, brandName already registered)
413	Payload Too Large	Request body or file exceeds configured size limit
422	Unprocessable Entity	Input validation failed — errors array present in response
429	Too Many Requests	Rate limit exceeded — retryAfter field indicates wait time
500	Internal Server Error	Unexpected server error — logged, safe message returned to client
503	Service Unavailable	Database connection timeout

8. Security Requirements

8.1 Security Layers Summary

Layer	Implementation	Attack Prevented
Security Headers	Helmet.js (15+ headers)	XSS, clickjacking, MIME sniffing, content injection
CORS	Origin whitelist validation	Unauthorized cross-origin API access
Rate Limiting	express-rate-limit (6 tiers)	Brute force attacks, DDoS, API scraping
NoSQL Injection	express-mongo-sanitize	MongoDB operator injection ($gt, $ne, $where)
XSS Escaping	.escape() on text fields	Script injection in stored content
HTTP Param Pollution	express-hpp	Array injection via duplicate query params
Password Hashing	bcrypt (salt rounds: 12)	Plain-text password exposure on DB breach
Token Security	JWT + httpOnly cookies	XSS token theft, CSRF attacks
Input Validation	express-validator on all routes	Malformed data, field injection
Ownership Checks	Controller-level verification	Unauthorized resource modification
RBAC	Role + permission middleware	Privilege escalation
Env Validation	Startup validation (fail-fast)	Missing secrets in production deployment
select: false	Mongoose schema config	Accidental password/token exposure in responses
Soft Delete	isActive flag pattern	Irreversible data loss, forensic capability
Token Rotation	Refresh token reuse detection	Refresh token theft and replay attacks

8.2 Rate Limit Configuration

Route Group	Max Requests	Window	Special Behavior
Global (all routes)	100	15 minutes	Applied first before route matching

Auth login/register	10	15 minutes	skipSuccessfulRequests: true — only failed attempts counted
Admin login	5	15 minutes	Strictest limit — high-value target
File upload	20	1 hour	Per partner IP to prevent CDN cost abuse
Search endpoints	30	1 minute	Prevent automated scraping
AI recommendation	20	1 minute	Compute-intensive endpoints
Interaction tracking	60	1 minute	High-frequency frontend calls — higher limit

8.3 JWT Token Specification

Token Type	Secret Env Var	Expiry	Payload Fields	Storage
User Access	JWT_SECRET	15 min	{ id, role: undefined }	Authorization header (Bearer)
User Refresh	JWT_REFRESH_SECRET	7 days	{ id }	httpOnly cookie: refreshToken
Partner Access	JWT_SECRET	15 min	{ id, role: 'partner', brandName }	Authorization header (Bearer)
Partner Refresh	JWT_REFRESH_SECRET	7 days	{ id, role: 'partner' }	httpOnly cookie: partnerRefreshToken
Admin Access	JWT_SECRET	15 min	{ id, role: 'admin', adminRole }	Authorization header (Bearer)
Admin Refresh	JWT_REFRESH_SECRET	7 days	{ id, role: 'admin' }	httpOnly cookie: adminRefreshToken

9. AI Recommendation System Specification

9.1 Algorithm Type

The Drip AI recommendation system implements a Content-Based Filtering approach combined with implicit behavioral signals. This is the same core algorithm used by TikTok, Netflix, and Spotify before neural network layers are added. No external AI services or machine learning libraries are required.

> *Industry Note: Content-based filtering is the most appropriate algorithm for a cold-start fashion platform with limited initial data. It does not require large datasets or model training. Recommendations improve incrementally with each user interaction.*

9.2 Interaction Weight System

Every user interaction updates the categoryScores Map on their User document using MongoDB atomic $inc operations. Weights are calibrated based on the strength of purchase intent signal:

Interaction	Weight	Rationale
view	0.5	Weak signal — user may have scrolled past without genuine interest
like	2.0	Moderate signal — active positive response to content
comment	2.5	Strong signal — user engaged enough to type a response
bookmark	3.0	Strong signal — user intends to return and possibly purchase
share	3.5	Very strong signal — user promotes content to their network
order	5.0	Strongest possible signal — user committed money to this category

9.3 Outfit Scoring Formula

Each active outfit is scored against the current user profile using the calculateOutfitScore() pure function (no database calls):

Factor	Max Points	Calculation Method
Category Score Match	40	(userCatScore / maxCatScore) × 40 — normalized to prevent single-category dominance
Followed Partner Bonus	20	Binary: +20 if outfit's partner is in user's follows list, else 0
Tag Overlap	15	(matchingTags / userPreferences.length) × 15 — rewards outfits matching user's style keywords

Direct Category Preference	10	Binary: +10 if outfit category is in user's stylePreferences array
Trending Score	10	min((views×0.3 + likes×0.7) / 100, 1) × 10 — rewards popular content
Freshness Bonus	5	<1 day: 5pts, 1–3 days: 3pts, 3–7 days: 1pt, >7 days: 0pts
TOTAL	100	Sum of all factors. Higher score = higher position in personalized feed

9.4 Cold Start Solution — Style Quiz

New users with no interaction history receive no personalized recommendations. The style quiz endpoint (POST /api/ai/style-quiz) solves this by mapping survey answers to initial categoryScore values, immediately enabling personalized feed on first login.

Example mapping — answer 'casual' to preferred_style question:

- casual → categoryScores.casual += 10, categoryScores.streetwear += 5
- formal → categoryScores.formal += 10, categoryScores.luxury += 5
- ethnic → categoryScores.ethnic += 10

Multiple answers compound: a user answering 'casual' + 'daily_wear' + 'mid_range' receives starting scores of casual:17, streetwear:10, formal:2 — immediately producing a meaningfully personalized feed.

10. Real-Time System Specification

10.1 Socket.io Room Architecture

Socket.io uses room-based delivery to ensure events reach only their intended recipients. No broadcasting to all connected clients is used in production scenarios.

Room Name	Joined By	Receives Events
user:{userId}	Authenticated users on connect	new_notification, new_message, messages_read, message_deleted
partner:{partnerId}	Authenticated partners on connect	new_notification, new_message, messages_read, message_deleted
outfit:{outfitId}	Any client (join_outfit_room event)	Reserved for future live comment events
conv:{conversationId}	Chat participants (join_conversation event)	Reserved for in-conversation typing indicators

10.2 Server-Emitted Events

Event Name	Target Room	Payload	Trigger
new_notification	user:id or partner:id	{ _id, type, title, message, data, isRead, createdAt }	Like, comment, follow, order, new outfit
new_message	receiver's personal room	{ _id, conversationId, sender, text, messageType, outfitSnapshot, createdAt }	POST /api/chat/:otherPartyId
messages_read	sender's personal room	{ conversationId, readBy }	GET /api/chat/:otherPartyId (auto read-mark)
message_deleted	receiver's personal room	{ messageId, conversationId }	DELETE /api/chat/message/:messageId
user_typing	receiver's personal room	{ conversationId, isTyping: true/false }	Client emits 'typing' or 'stop_typing'

10.3 Client-Emitted Events

Event Name	Payload	Server Action
join_outfit_room	outfitId (string)	socket.join('outfit:' + outfitId) — logs join

leave_outfit_room	outfitId (string)	socket.leave('outfit:' + outfitId)
join_conversation	conversationId (string)	socket.join('conv:' + conversationId) — logs join
leave_conversation	conversationId (string)	socket.leave('conv:' + conversationId)
typing	{ conversationId, receiverRoom }	Forward user_typing(isTyping:true) to receiverRoom
stop_typing	{ conversationId, receiverRoom }	Forward user_typing(isTyping:false) to receiverRoom

10.4 Connection Authentication

Socket.io connections pass the JWT access token in the handshake auth object:

- socket.handshake.auth.token — primary location
- socket.handshake.headers.authorization (Bearer prefix stripped) — fallback

The Socket.io auth middleware verifies the token, finds the corresponding User or FashionPartner document, and attaches it to the socket instance. Invalid or missing tokens result in an unauthenticated connection — the socket connects but joins no personal room and receives no personal events.

11. Integration Requirements

11.1 ImageKit CDN Integration

Specification	Value
SDK	imagekit npm package v4.x
Authentication	publicKey + privateKey from environment variables
Upload method	Base64 encoded buffer (multer memoryStorage)
Allowed image types	image/jpeg, image/jpg, image/png, image/webp, image/gif
Allowed video types	video/mp4, video/mpeg, video/quicktime, video/x-msvideo, video/webm
Max image size	5 MB
Max video size	100 MB
Folder structure	/drip/outfits/videos — outfit reels; /drip/outfits/images — outfit photos; /drip/avatars — user profiles; /drip/logos — partner logos; /drip/covers — partner cover images; /drip/misc — other uploads
Deletion	Permanent deletion via fileId stored at upload time. Called on outfit deletion and account cleanup.

11.2 Stripe Payment Integration

Specification	Value
SDK	stripe npm package v14.x
Authentication	STRIPE_SECRET_KEY from environment variables
Processing currency	USD (PKR amount stored in metadata.actualAmountPKR)
Minimum charge	USD $0.50 (Stripe minimum — enforced in service layer)
Payment flow	1. Checkout → createPaymentIntent → clientSecret returned; 2. Frontend collects card via Stripe.js; 3. Stripe webhook payment_intent.succeeded → order confirmed
Webhook verification	STRIPE_WEBHOOK_SECRET used with stripe.webhooks.constructEvent() to verify signature
COD orders	Bypass Stripe entirely — order created with paymentStatus: unpaid
Refunds	createRefund(paymentIntentId) available for future customer service feature

12. Constraints and Assumptions

12.1 Technical Constraints

Constraint	Value	Reasoning
Node.js version	18+	ES modules, modern async patterns, performance improvements
MongoDB version	6+	Atlas free tier compatibility, aggregation pipeline features
Max images per outfit	5	Storage cost control and page load performance
Max tags per outfit	10	MongoDB text index optimization
Max cart quantity per item	10	Prevent inventory hoarding
Comment nesting	1 level	UX simplicity — avoids Reddit-style deep threading
Message delete window	24 hours	Industry standard — WhatsApp model
AI feed pool	200 outfits	Balance between recommendation quality and response time
Access token lifetime	15 minutes	Security best practice — minimizes token theft window
Refresh token lifetime	7 days	Balance between security and user experience
Max page size	50 (general), 20 (feed)	Performance and data transfer limits
Max message length	1000 characters	Prevent abuse, encourage concise communication
Max comment length	500 characters	Same as above

12.2 Business Assumptions

1. Users have stable internet connections capable of streaming video content from ImageKit CDN.
2. ImageKit handles all video transcoding, thumbnail generation, and adaptive streaming delivery.
3. Stripe handles all payment card security and PCI DSS compliance — card data never touches the Drip backend.
4. MongoDB Atlas handles data replication, automated backups, and point-in-time recovery.
5. Fashion Partner accounts represent legitimate business entities — admin manual review of applications is the primary fraud prevention mechanism.
6. The platform initially serves the Pakistani market (PKR as display currency) but processes payments in USD via Stripe.

7. The frontend React application implements Socket.io client connection management, including reconnection logic on token refresh.

12.3 Known Limitations

Limitation	Impact	Mitigation
AI cold start for new users	New users receive non-personalized feed initially	Style quiz endpoint seeds initial categoryScores on onboarding
Stripe PKR limitation	PKR not natively well-supported by Stripe test mode	Amounts processed in USD; actual PKR stored in order and Stripe metadata
Real-time requires connection	Offline users miss real-time events	All notifications persisted to MongoDB — delivered on next GET /api/notification
Large follower notifications	Bulk notifications to thousands of followers may have slight delay	Promise.allSettled ensures partial delivery; future enhancement: message queue (Bull/Redis)
Single-server Socket.io	Socket.io rooms only work on single server instance	For horizontal scaling: implement Redis adapter (socket.io-redis)
AI scoring pool limit	Outfits beyond 200 never scored for personalized feed	Acceptable for MVP; future: pre-compute scores and cache in Redis

End of Software Requirements Specification

Drip Fashion Reels Platform — Backend API — Version 1.0 — April 2026

Hassan Shehzad